

Get started with the course

Library

- A reusable collection of code

Python versus other programming languages

Python is one of the most popular programming languages for data professionals, which makes it a great addition to your data analytics toolbox! As we've previously investigated, Python's use of syntax to communicate commands and perform tasks mirrors spoken language. This makes Python a much easier programming language to learn. Python's structure is similar to many other programming languages, but there are some key differences to consider as well.

In this reading, you'll learn how Python compares to other programming languages data professionals use, including R, Java, and C++.

Five considerations of programming languages

Python isn't the only programming language used for data analysis, but it is one of the most widely used and most powerful. Many data professionals even use more than one programming language. Every language has benefits and drawbacks. For the purposes of this course, examine the following considerations: speed, approachability, variables, data science focus, and programming paradigm.

Speed

There are many factors that contribute to the speed of a program's execution, including compile time, runtime, hardware, installed dependencies, and the efficiency of the code itself. In general, low-level programming languages are faster, but they're more difficult to learn and work with.

Approachability

Approachability refers to how easy it is for new learners to start using a language. Learning new programming languages can be challenging depending

on their syntax and overall structure. The **syntax** is the structure of code words, symbols, placement, and punctuation. Semantics builds meaning into those structures by using variables and objects. Additionally, those variables help add flexibility to the programs and objects where data is housed.

Variables

Information in code is stored in variables. A **variable** is a named container which stores values in a reserved location in the computer's memory. The way a programming language uses variables will have an effect on a system's core operations or kernel speed. Some languages use static variables to maintain a value throughout the entire run of a program. Others approach variables as dynamic, allowing values to be determined when a program is run. Some languages even allow declarative variables, which enable a program to determine where a variable should be placed.

Data science focus

Programming languages have individual characteristics and can better serve different tasks in data analysis; this means programmers often use them for specific data science tasks.

Programming paradigm

Programming languages can be object-oriented, functional, or imperative. Object-oriented programming languages are modeled around data objects. Functional programming languages are modeled around functions. Imperative languages are modeled around code statements that can alter the state of the program itself.

Programming language comparisons

Python, R, Java and C++ are four of the most commonly used programming languages for data analysis. The following chart compares them using five considerations: speed, accessibility, variable, data science focus, and programming paradigm.

Features by Software	Python	R	Java	C++
Speed	Slower	Depends on configuration and add-ons	Faster	Very fast

Approachability	Easy to learn	Complex	Easy to learn	Complex
Variable	Dynamic	Dynamic	Static	Declarative
Data science focus	Machine learning and automated analysis	Exploratory data analysis and building extensive statistical libraries	Used across projects with open-source assets	Not as widely used but very powerful implementations
Programming Paradigm	Object-oriented	Functional language	Object-oriented	Multi-paradigm (imperative & object-oriented)

Key takeaways

There are a number of different programming languages that can be used for data analysis. Each language has its own benefits and drawbacks. Learning to work with different languages will give you the opportunity to broaden your data skills and access new tools for your analysis. However, in this certificate program, Python will be your sole focus. As mentioned previously, Python is an easy to learn, object-oriented programming language that engages dynamic variables; though it sometimes requires a longer time to execute, it is a great tool for machine learning and automated analysis.

Jupyter Notebooks

Cells

- The modular code input/output fields into which Jupyter Notebooks are partitioned

How to use Jupyter Notebooks

Jupyter Notebook is an open-source web application for creating and sharing documents containing live code, mathematical formulas, visualizations, and text. This is a great tool to develop and present code in a standardized text block format that is interactive and shareable. You can create code, mathematical formulas, data visualizations, and even freestyle text—all in Jupyter notebooks!

You will be using Jupyter notebooks to write, execute, and present your own code throughout this program. This reading will guide you through using your

own notebook. Note, however, that for this certificate program you do not need to download any software. You can complete all activities with the tools provided on the Coursera platform.

Jupyter Notebook

You can access Jupyter Notebook directly from your browser or download the desktop application onto your device to work with over 100 programming languages, including some you might already know like R and Python. There is [JupyterLab](#), which is the full suite of tools for working with computational notebooks. There is also [Jupyter Notebook](#), which is a more streamlined and simplified tool that nonetheless offers powerful ways to perform interactive computing. Again, for this certificate program, we recommend working within the Jupyter Notebook interface provided by Coursera. Activities that use Jupyter notebooks will be labeled as labs, and you will find relevant instructions for each activity on its landing page.

Why Jupyter Notebook?

Notebooks are particularly useful for working with data. Here are some ways that Jupyter notebooks excel:

1. **Modular/interactive computing:** You can write and execute individual chunks of code in small, manageable chunks, which are called cells. You can run a cell without necessarily having to run the whole notebook. This is especially helpful for data exploration and experimentation. Cells are also helpful with debugging, because they provide a user-friendly way to make a mistake, notice that you made the mistake, and iterate back to correct your mistake, without having to re-execute a whole script.
2. **Integration of code and documentation:** Notebooks allow you to combine code, textual explanations, and visualizations like charts, graphs, and tables—all in a single document.
3. **Support for multiple languages:** The Advanced Data Analytics program will use Python, but Jupyter notebooks support many other languages, making them powerful and versatile.
4. **Data exploration and analysis:** The notebook simplifies working with data by offering tools to load, clean, analyze, and examine it in an elegant interface.

5. **Cloud-based services:** Many cloud computing platforms host Jupyter notebooks, which makes it easy to run and share notebooks without setting up a local environment. This is very useful for collaboration.
6. **Libraries and extensions:** There is a rich ecosystem of extensions and plugins that enhance functionality for whatever type of project you're working on.

How to use Jupyter notebooks

Once you've opened a Jupyter notebook, it's time to use it! Here are some tips to get started.

Command/edit mode

Notebooks have two working modes: command mode and edit mode. Command mode is used to interact with the notebook as a whole and perform actions like adding, moving, and deleting cells. Edit mode is used to type code or markdown text into a particular cell.

Command mode is indicated by a blue bar on the left side of the current cell.



Edit mode is indicated by a green bar on the left and also a thin green border around the active cell.



To enter into edit mode, simply click into a cell to insert your cursor there or use the navigation arrows on your keyboard to select a cell and press Enter. To revert back to command mode, click anywhere outside the cell or press the escape key.

Markdown mode

Jupyter notebooks allow you to toggle cells between coding mode and Markdown mode. Markdown is a markup language that lets you add formatting elements to plain text. It's useful because it's ubiquitous, future-proof, and platform independent. In Jupyter notebooks, Markdown text is used to provide written explanations, analysis, and context to explain the code and its output. In

the following example, all of the text above **In [41]:** was written and formatted using Markdown.



To toggle between code and Markdown mode, go to the menu at the top of the page and click where it says "Code." Then, select "Markdown" from the dropdown menu that appears.



For more information about how to use Markdown, refer to [Markdown guide for basic syntax](#) and the [Markdown guide for extended syntax](#).

Common actions

Most actions can be performed using both a mouse/graphic interface and keyboard shortcuts. Here are some of the most common actions.

Add a new cell

- Click on Insert in the menu bar at the top of the notebook. Options are to insert a new cell above or below the current cell.
- Keyboard shortcuts (while in command mode):
 - **a**: Insert a cell above the current cell
 - **b**: Insert a cell below the current cell

Delete a cell

- Use command mode to select a cell or group of cells.
- Click on Edit in the menu bar at the top of the notebook and select Delete Cells from the dropdown menu.
- Keyboard shortcut (while in command mode):
 - **dd** (press D two times)

Move a cell

- Use command mode to select a cell or group of cells.

- Click on the up arrow button or down arrow button in the menu bar at the top of the notebook to move the selected cell(s) up or down

Run a cell

- Select a cell and click the **Run** button in the menu bar at the top of the notebook.
- Keyboard shortcuts:
 - **Ctrl + Enter**: Run selected cell
 - **Shift + Enter**: Run selected cell and select next cell
 - **Alt + Enter**: Run selected cell and insert new cell below
- You can run cells from both command mode and edit mode

Press **h** while in command mode for a pop-up window with all available keyboard shortcuts. You can also check out [Jupyter Notebook interface components](#) for more detailed descriptions of various notebook features.

Troubleshooting

You will use Jupyter notebooks throughout the Advanced Data Analytics certificate program. At times, you might encounter difficulty accessing or running the notebook. Here are some troubleshooting steps to help you if this happens.

Browser compatibility

Make sure your internet browser is updated regularly. It is best to use the latest version of Google Chrome, Firefox, or Microsoft Edge. If your browser is outdated or you are using a browser that is not supported by Coursera, you may encounter a problem. If your browser is up to date and you are using one of the browsers listed above and still encountering problems, try restarting your browser or clearing your browser's cache and cookies. You can also use incognito mode, which prevents your browser from storing cookies and other temporary data.

Internet connection

Coursera requires a stable internet connection. If you are experiencing problems starting or running a Jupyter notebook, your internet connection may be slow or

unreliable. Some signs of an unstable internet connection may be pages failing to load, freezing labs, or the inability to type or enter commands within the lab environment.

Pro Tip: If you are unable to complete a lab on one device, try using another device.

Troubleshooting steps

To summarize, here are the troubleshooting steps to try if you encounter a problem with Jupyter notebooks in Coursera.

1. Make sure you are using the latest version of a supported browser: Google Chrome, Firefox, or Microsoft Edge.
2. Restart your browser and clear your browser's cache and cookies. You can also use incognito mode.
3. Check your internet connection and make sure it is stable. You can try restarting your router and modem to regain a stable connection.
4. Try restarting the lab again.

If all this fails, it's possible that Coursera is performing maintenance or experiencing a service interruption. In that case, wait a little while and try again.

Key takeaways

Jupyter Notebook provides a coding platform where you can develop and debug your own code. Knowing how to use and interact with notebooks will prepare you for upcoming activities where you will try out new Python skills and prepare for the end-of-course project. Python will be a great tool in your toolkit—it will open up more advanced analytics tools like machine learning and automated analysis. And, using Jupyter Notebook will be a great way to build your Python knowledge!

Resources for more information

- [Jupyter Notebooks interface training](#)
- [Jupyter software homepage](#)
- [Jupyter documentation](#)
- [Jupyter Notebooks cloud \(online\)](#)

- [Jupyter community forum](#)
- [Jupyter notebooks community forum](#)
- [Python community forum](#)
- [StackOverflow questions](#) (crowdsource forum to help solve problems)
- [Jupyter Notebooks installation](#)

Object-oriented programming

- A programming system that is based around objects, which can contain both data and code that manipulates that data

Object

- An instance of a class;
- A fundamental building block of Python

Class

- A class in an object's data type that bundles data and functionality together

Method

- A function that belongs to a class and typically performs an action or operation

Dot Notation

- How to access the methods and attributes that belong to an instance of a class

Core Python Classes

- Integers
- Floats
- Strings
- Booleans
- Lists
- Dictionaries

- Tuples
- Sets
- Frozensets
- Functions
- Ranges
- None
- Custom-defined

Attribute

- A value associated with an object or class which is referenced by name using dot notation

More about object-oriented programming

Note: This reading contains only a brief introduction to object-oriented programming. A more detailed discussion about the nuances of object oriented programming is beyond the scope of this course.

Previously, we identified object-oriented programming as a programming paradigm that is based around objects, which can contain both data and code that manipulates that data. You may recall that a class is an object's data type that bundles data and functionality together, and you've encountered some examples of this class-specific functionality in the form of methods and attributes. In this reading, you're going to learn more about object-oriented programming and how it works. Although this certificate program will not require you to define your own classes, having a basic understanding of how this process works will be very helpful when you encounter these concepts along your learning journey.

Review: Attributes and methods

Python classes are powerful and convenient because they come with built-in features that simplify common data analysis tasks. These features are known as attributes and methods.

- **Attribute:** A value associated with an object or class which is referenced by name using dot notation.

- **Method:** A function that belongs to a class and typically performs an action or operation.

A simpler way of thinking about the distinction between attributes and methods is to remember that attributes are *characteristics* of the object, while methods are *actions* or *operations*.

For example, if the class were Spaceship, then attributes might be:

name

kind

speed

tractor_beam

These attributes could be accessed by typing:

Spaceship.name

Spaceship.kind

Spaceship.speed

Spaceship.tractor_beam

Notice that these characteristics are accessed using only a dot.

On the other hand, methods of the Spaceship class might be:

warp()

tractor()

These methods could be used by typing:

Spaceship.warp()

Spaceship.tractor()

Notice that methods are followed by parentheses, and it's possible for them to take arguments. For example, **Spaceship.warp(7)** could change the speed of the ship to warp seven.

Defining classes with unique attributes and methods

Python lets you define your own classes, each with their own special attributes and methods. This helps all different kinds of programmers to build reusable code that makes their work more efficient. You can even build the Spaceship class mentioned previously. The example, here, demonstrates how to do this.

Note: The following code block is not interactive.

```
class Spaceship:
    # Class attribute
    tractor_beam = 'off'

    # Instance attributes
    def __init__(self, name, kind):
        self.name = name
        self.kind = kind
        self.speed = None

    # Instance methods
    def warp(self, warp):
        self.speed = warp
        print(f'Warp {warp}, engage!')

    def tractor(self):
        if self.tractor_beam == 'off':
            self.tractor_beam = 'on'
            print('Tractor beam on.')
        else:
            self.tractor_beam = 'off'
            print('Tractor beam off')
```

For this course you don't have to learn the syntax to create classes. Just notice that the class itself is defined first, and then indented beneath it are the attributes and methods. This is what it means when an attribute or method "belongs to a class." Attributes and methods are defined in the code for that class.

A class is like a blueprint for all things that share characteristics and behaviors. In this case, the class is Spaceship. There can be all different kinds of spaceships. They can have different names and different purposes. Whenever you create an object of a given class, you're creating an **instance** of that class. This is also known as **instantiating** the class. In the code above, every time you instantiate an object of the Spaceship class it will start with its tractor beam set to off. The tractor beam is a class attribute. All instances of the Spaceship class

have one. There are also instance attributes. These are attributes that you can assign when you instantiate the object.

```
# Create an instance of the Spaceship class (i.e. "instantiate")
ship = Spaceship('Mockingbird','rescue frigate')

# Check ship's name
print(ship.name)

# Check what kind of ship it is
print(ship.kind)

# Check tractor beam status
print(ship.tractor_beam)
```

The next block of code uses the **warp()** method to set the warp speed to seven. Then it checks the current speed of the ship using the speed attribute.

```
# Set warp speed
ship.warp(7)

# Check speed
ship.speed
```

This final block of code uses the **tractor()** method to toggle the tractor beam. Then it checks the current status of the tractor beam using the **tractor_beam** attribute.

```
# Toggle tractor beam
ship.tractor()

# Check tractor beam status
print(ship.tractor_beam)
```

This is just a basic example meant to demonstrate some of the fundamental ways that classes, attributes, and methods work and how they relate to each other, but classes can be very complex and have many attributes and methods. Depending on the work you're doing as a data professional, knowledge about object-oriented programming will be helpful as you define your own classes,

attributes, and methods to investigate the patterns, relationships, and meaning data holds.

Key takeaways

Classes comprise the core objects of Python, which is why Python is known as an object-oriented language. Class objects are powerful because they contain unique tools designed specifically for that class packaged within them. Methods are functions that belong to a class; they perform actions or operations, and they use parentheses. Attributes are values or characteristics associated with a class or class instance; they do not use parentheses. And, while there are many classes, attributes, and methods pre-built into Python, there is a high level of customization offered in the object-oriented programming paradigm.